République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
ECOLE SUPÉRIEURE EN INFORMATIQUE
8 Mai 1945 - Sidi-Bel-Abbès

الجمهورية الجزائرية الديمقراطية الشعبية
وزارة التعليم العالي والبحث العلمي
المدرسة العليا للإعلام الآلي
8 ماي 1945 - سيدي بلعباس

Advanced Visualization Plots

Presented By Pr. Nabil KESKES.

2023-2024

.

PLAN

- Waffle Charts
- World Cloud
- Regression Plots

1. Waffle Chart

- ✓ A bar chart is preferable to a pie chart because **it will convey your arguments more succinctly and simply.**
- ✓ Bar charts do not effectively convey **the part-to-whole comparison**, which is a **pie chart's main advantage**
- ✓ **Waffle charts** are a much better way of showing the same sort of data that a pie chart would. With a waffle chart, you get **a more segmented** view of the data

1. Waffle Chart

1.2 Definitions

A waffle chart is a 10x10 grid in which each cell represents 1 percentage point, summing up to a total of 100%. It is used to show different categories that add up to a whole of a phenomenon, just like pie charts. Use a waffle chart when you are comparing **more than 5 categories.**

a.Example 1

Example 1 shows how to create a waffle plot an array that represents count of observations for different groups, so each group will have as many tiles as its corresponding value.

```
from pywaffle import Waffle
import matplotlib.pyplot as plt
# Data
value = {'G1': 12, 'G2': 22, 'G3': 16, 'G4': 38, 'G5': 12}
# Waffle chart
plt.figure(
    FigureClass = Waffle,
    rows = 10,
    columns = 10,
    values = value,
    vertical = True,
    title = {"label": "Title of the chart", "loc": "right", "size": 15},
    legend = {'loc': 'upper left', 'bbox_to_anchor': (1, 1)})
# plt.show()
plt.show()
```

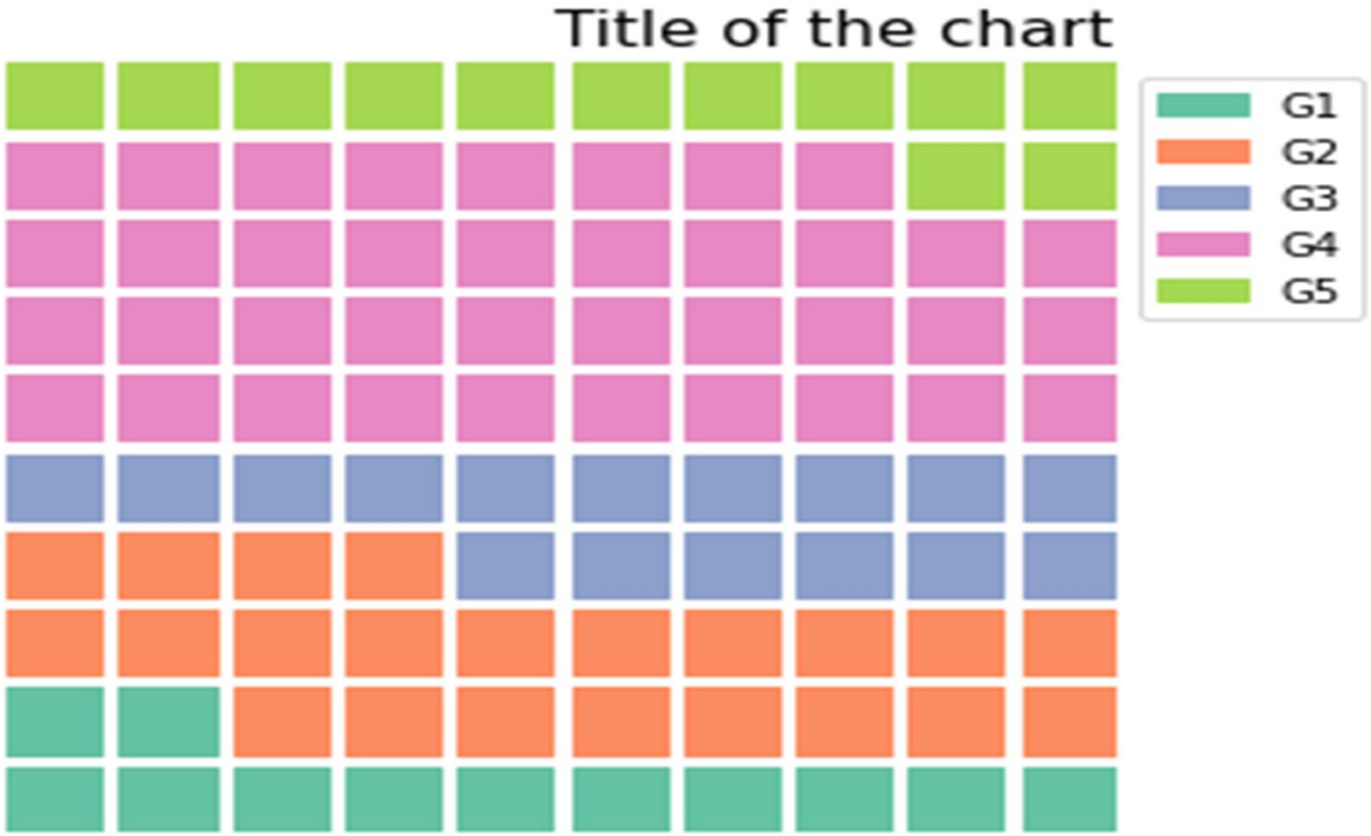


Figure 1.1: Waffle Chart Plotting

b.Example 2

If you want **to combine several waffle charts** in one single figure you can accomplish it following the example below.

```
from pywaffle import Waffle
import matplotlib.pyplot as plt
import pandas as pd
# Data
df = pd.DataFrame({
        'labels': ['G1', 'G2', 'G3', 'G4'],
        'A': [10, 15, 25, 30],
        'B': [5, 40, 12, 16]}).set_index('labels')
```

```
# Waffle subfigure
fig = plt.figure(
    FigureClass = Waffle,
    plots = {
        211: {
            'values': df['A'],
            'labels': [f"{k} ({v})" for k, v in df['A'].items()],
            'legend': {'loc': 'upper left', 'bbox_to_anchor': (1.05, 1), 'fontsize': 8},
            'title': {'label': 'Chart A', 'loc': 'left', 'fontsize': 12}
        },
        212: {
            'values': df['B'],
            'labels': [f"{k} ({v})" for k, v in df['B'].items()],
            'legend': {'loc': 'upper left', 'bbox_to_anchor': (1.2, 1), 'fontsize': 8},
            'title': {'label': 'Chart B', 'loc': 'left', 'fontsize': 12}
        }
    },
    rows = 10,
    cmap_name = "Set1")
fig.suptitle('Title for all plots', fontsize = 12, fontweight = 'bold')
# plt.show()
```

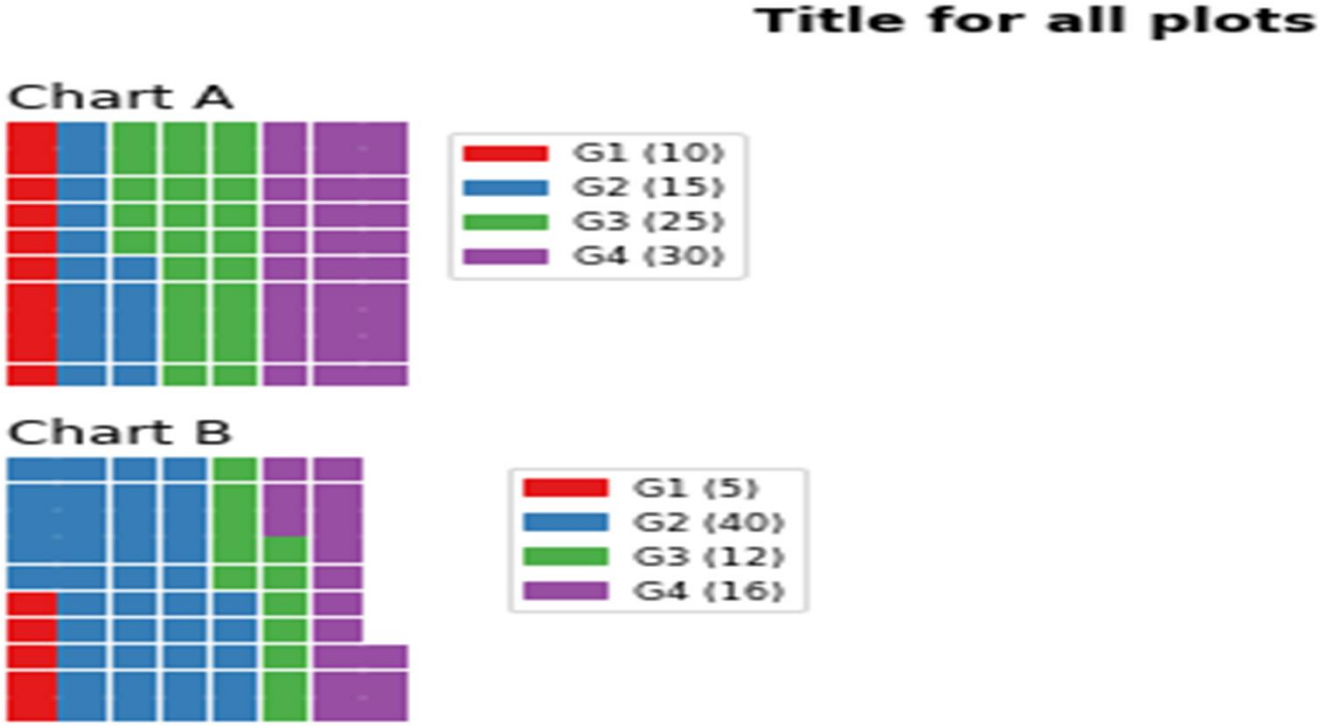


Figure 1.2: two Waffle Charts in on single figure

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
ECOLE SUPÉRIEURE EN INFORMATIQUE
8 Mai 1945 - Sidi-Bel-Abbès

الجمهورية الجزائرية الديمقراطية الشعبية
وزارة التعليم العالي والبحث العلمي
المدرسة العليا للإعلام الآلي
8 ماي 1945 - سيدي بلعباس

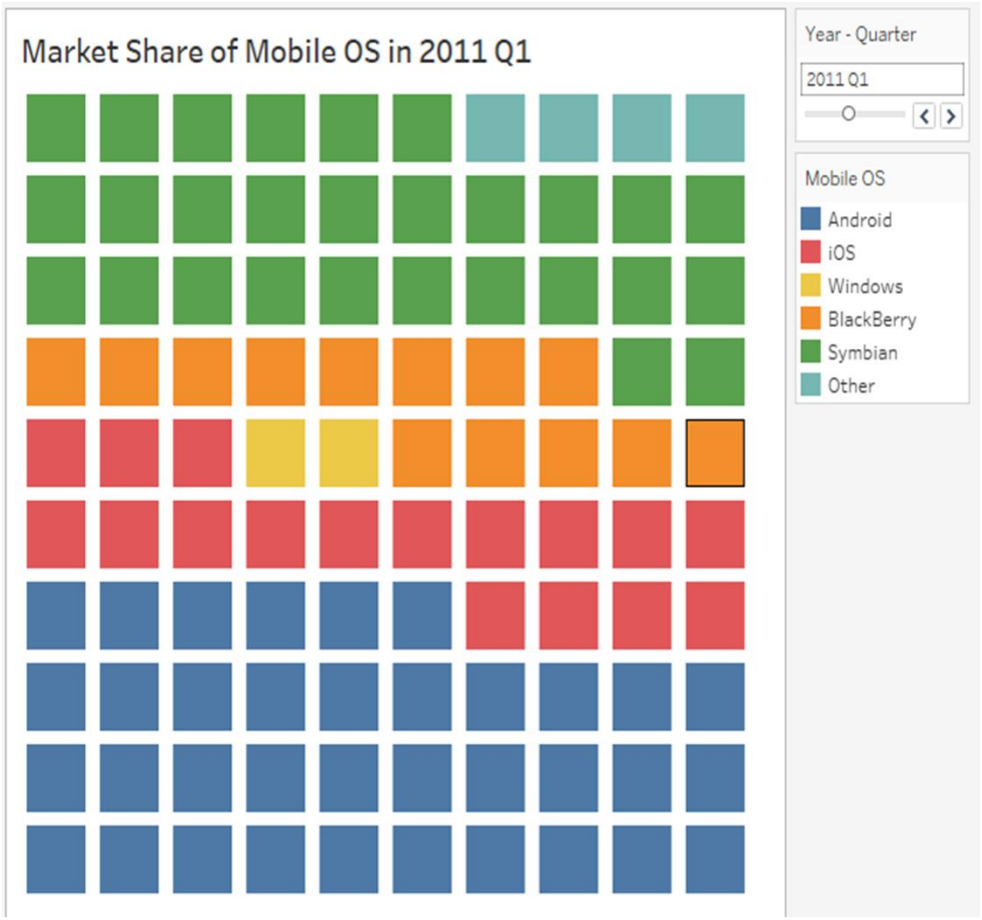


Figure 1.3: Market Share of Mobile interpretation

1.3 Interpretation:

This all-in-one waffle chart gives us a clear idea of each Mobile OS contribution to a whole. Even without labels, we can know the approximate proportions by counting the cells (one cell represents 1%).

Since the waffle chart can only observe one point in time, we use Filter to analyze over time. For example, before 2009, Android had almost zero market share. But *Android*'s market share rose rapidly to more than 85% in 2017.

2. World Cloud

Word clouds are a useful visualization tool. They show the most frequent words in a text, where **the relative size** of the word correlates **with frequency.**

Word clouds are useful for at least two purposes:

- ✓ An initial **exploration of text** to discover which words are most numerous.

- ✓ In conjunction with **a topic model**, it creates visuals for each of the topics, where the **most representative words for each topic** are evident

2.1 Transform the most critical information into a word cloud.

Here's an example from USA using U.S. President Barack Obama's State of the Union Speech 2012:

Figure 2.1: Speech presented by world cloud

As you can see, words like **"American**," **"jobs," "energy"** and **"every"** stand out since they were used more frequently in the original text.

Now, compare that to the 2014 State of the Union address:

Figure 2.2: Speech presented by world cloud

You can easily see the similarities and differences between the two speeches at a glance. **"America" and "Americans"** are still major words, but **"help," "work,"** and **"new"** are more prominent than in 2012.

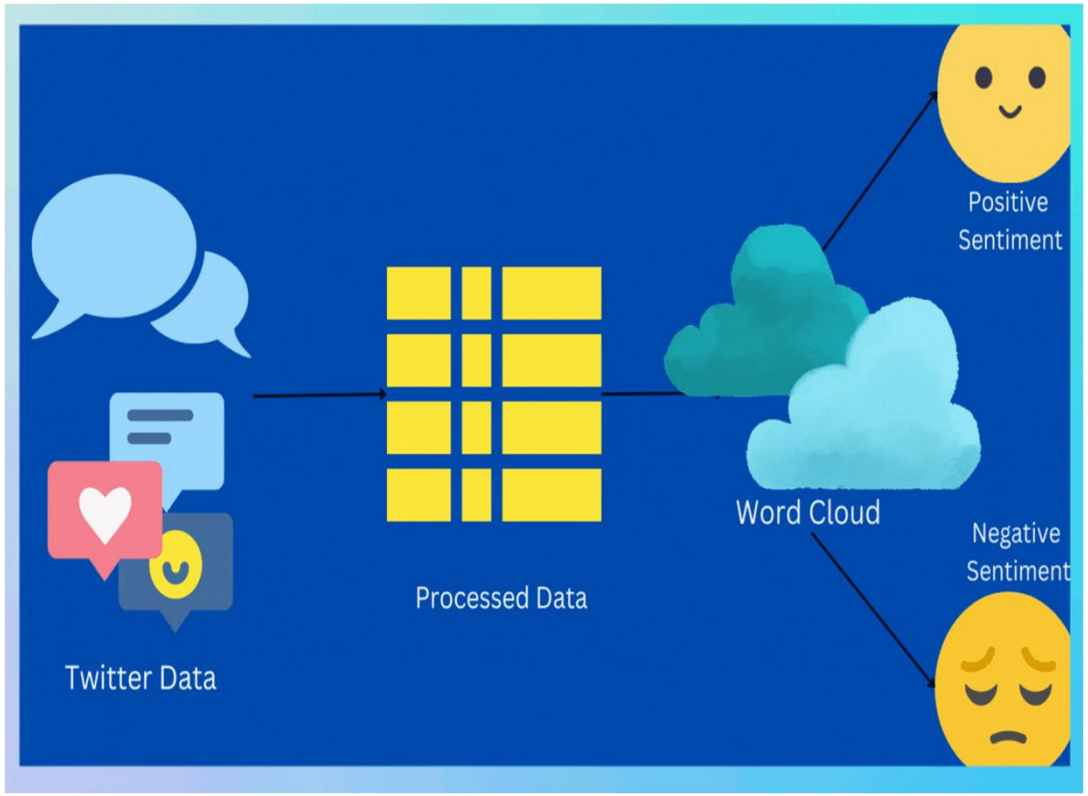
2.2 Quickening work Actions

That important marketing research report just came in at 5:00 p.m. The only problem? It's 50 pages long.

You'd love to read through it line by line, but you don't have enough hours in the day, and you're supposed to give a brief on the data the next morning. **This is where a word cloud generator can help.**

When you copy the text into the generator and let it do its job, you can see in seconds which **talking points appear the most frequently**. Then, you'll know where to start your search to hit the most important parts.

2.3 Word Cloud and Sentiment Analysis



We can break this down into four steps:

- ✓ Data extraction from Twitter using tweepy
- ✓ Data cleaning & processing
- ✓ **Visualization** using **wordcloud**
- ✓ Sentiment Analysis

Figure 2.3: Text mining

The idea is to build a word cloud which can give information about recession and not just repeat that word! Also, we do not want generic words such as ‘will’, ‘go’, ‘has’, ‘would’ etc. to appear in our word cloud. Nltk’s ‘stopwords’ provides a list of all such words, and we can exclude all of them from our ‘translated_text’.

Figure 2.4: Text mining

Per twitter data word cloud people, in the context of recession, are talking about **inflation**, **layoffs** and **jobs** which is sort of true! Particular focus on stock market.

Example

```
import matplotlib.pyplot as plt
from wordcloud import WordCloud
text = "Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat"

wc = WordCloud(width = 300, height = 300).generate(text)
# Remove the axis and display the data as image
plt.axis("off")
plt.imshow(wc, interpolation = "bilinear")

# plt.show()
```

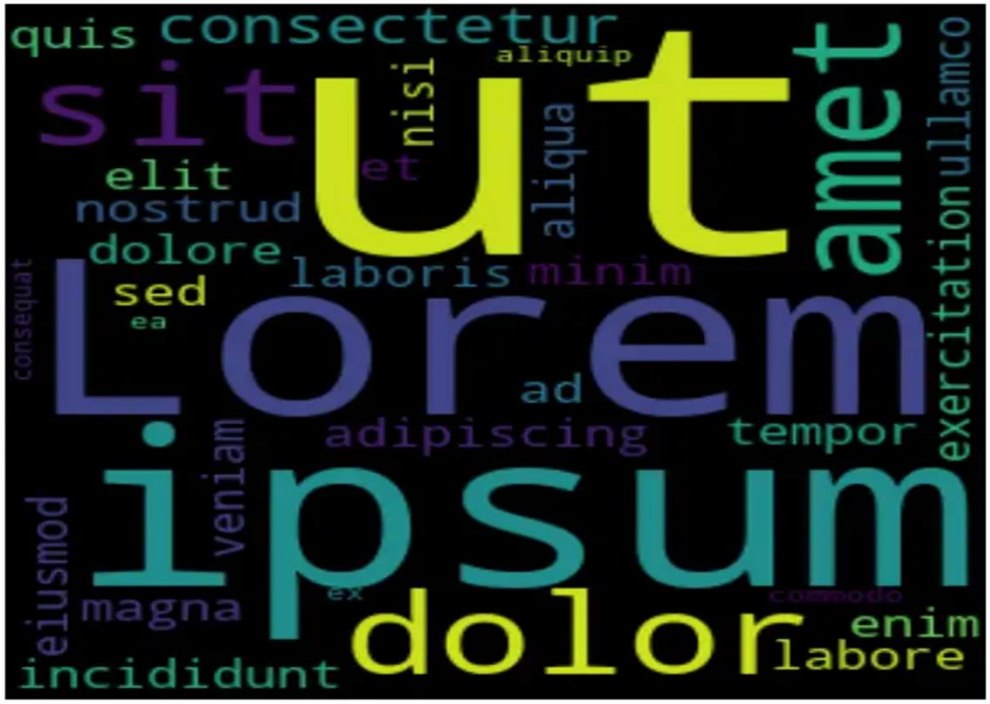


Figure 2.5: World Clouds plot

3. Regression Plots

- ✓ Many datasets contain multiple quantitative variables, and the goal of an analysis is often to relate those variables to each other.
- ✓ The regression plots in **seaborn** are primarily intended to add a visual guide that helps to emphasize patterns in a dataset during exploratory data analyses
- ✓ The goal of seaborn, however, is to make **exploring a dataset** through **visualization quick and easy.**

3.1 Functions to draw linear regression models

Two main functions in seaborn are used to visualize a linear relationship as determined through regression.These functions, regplot() and lmplot().

- ✓ Both functions draw **a scatterplot of two variables**, and then fit the regression model and plot **the resulting regression line** and a 95% confidence interval for that regression

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
df = pd.read_csv("c:/rep1/tips.csv")
sns.regplot(x="total_bill", y="tip", data=df)
sns.lmplot(x="total_bill", y="tip", data=df)
```

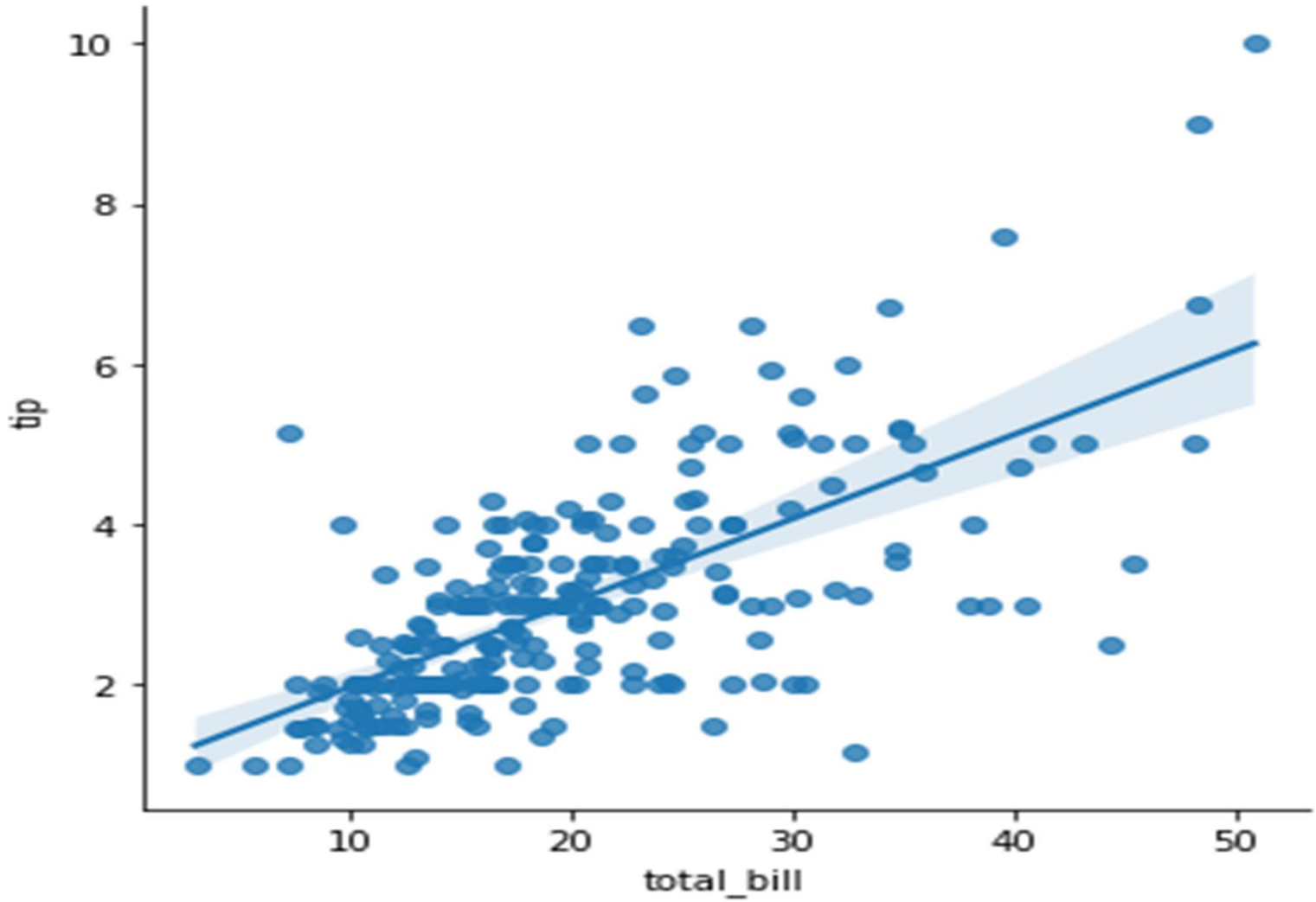


Figure 3.1: Linear regerssion plot

✓ It's possible to fit a linear regression when one of the variables takes **discrete values,** however, the simple scatterplot produced by this kind of dataset is **often not optimal.**

sns.lmplot(x="size", y="tip", data=df);

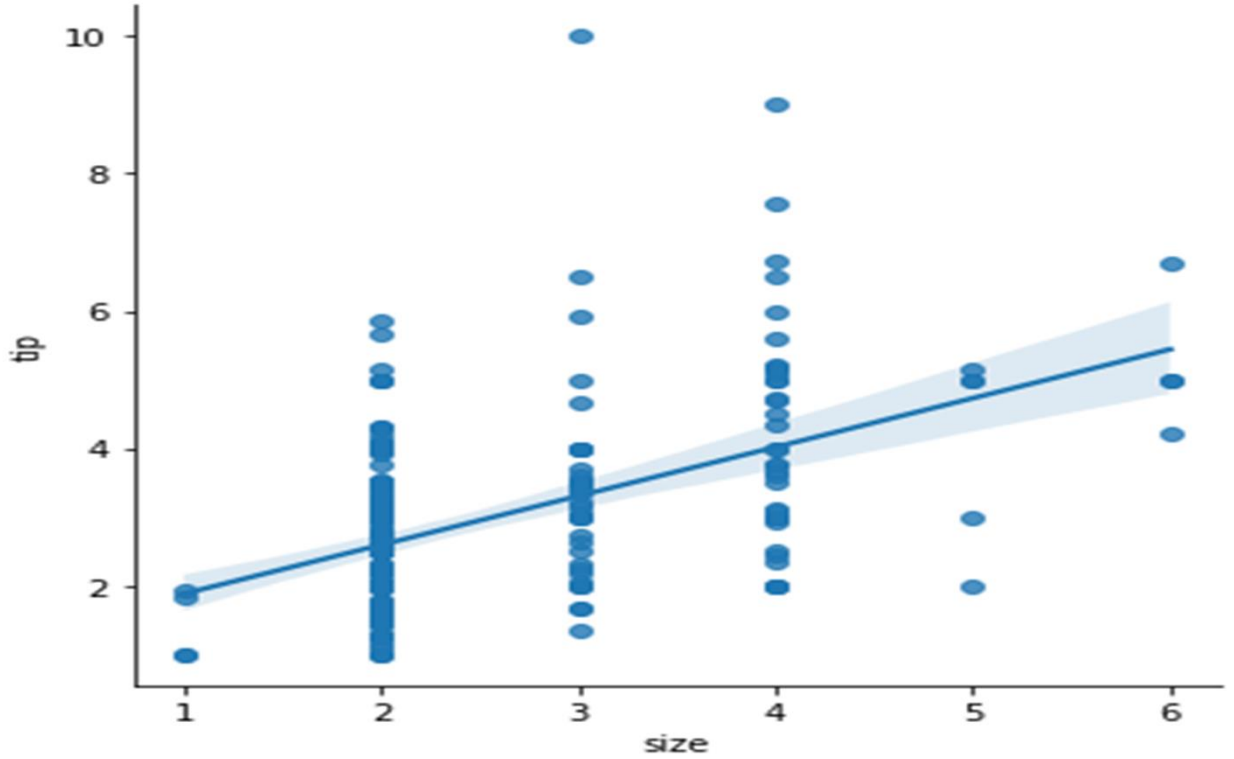


Figure 3.2: Linear regerssion plot with variable takes the discrete values

✓ One option is to add some random noise ("jitter") to the discrete values to make the **distribution of those values more clear**. Note that jitter is applied only to the scatterplot data and **does not influence** the regression line fit itself.

sns.lmplot(x="size", y="tip", data=df, x_jitter=.05);

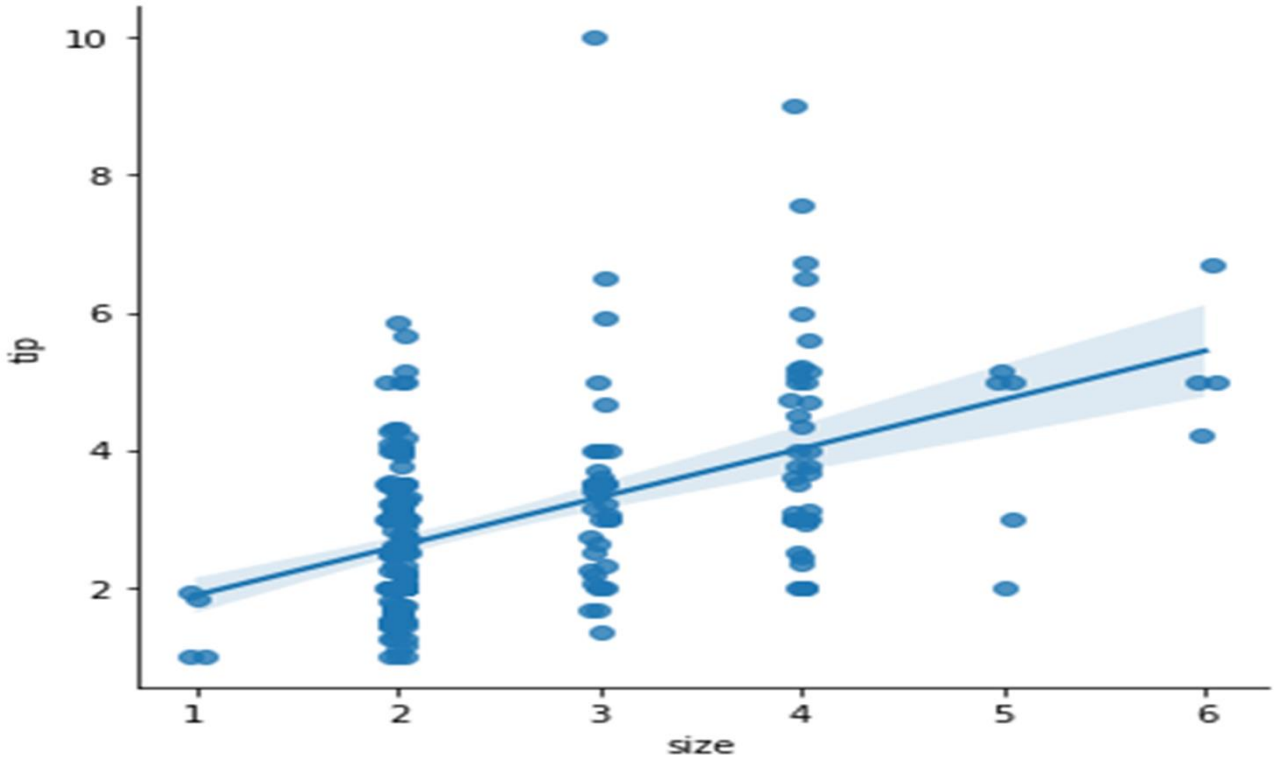


Figure 3.3: Linear regerssion plot with jitter

✓ A second option is to collapse over the observations in each discrete bin to plot an estimate of central tendency along with a confidence interval.

sns.lmplot(x="size", y="tip", data=df, x_estimator=np.mean);

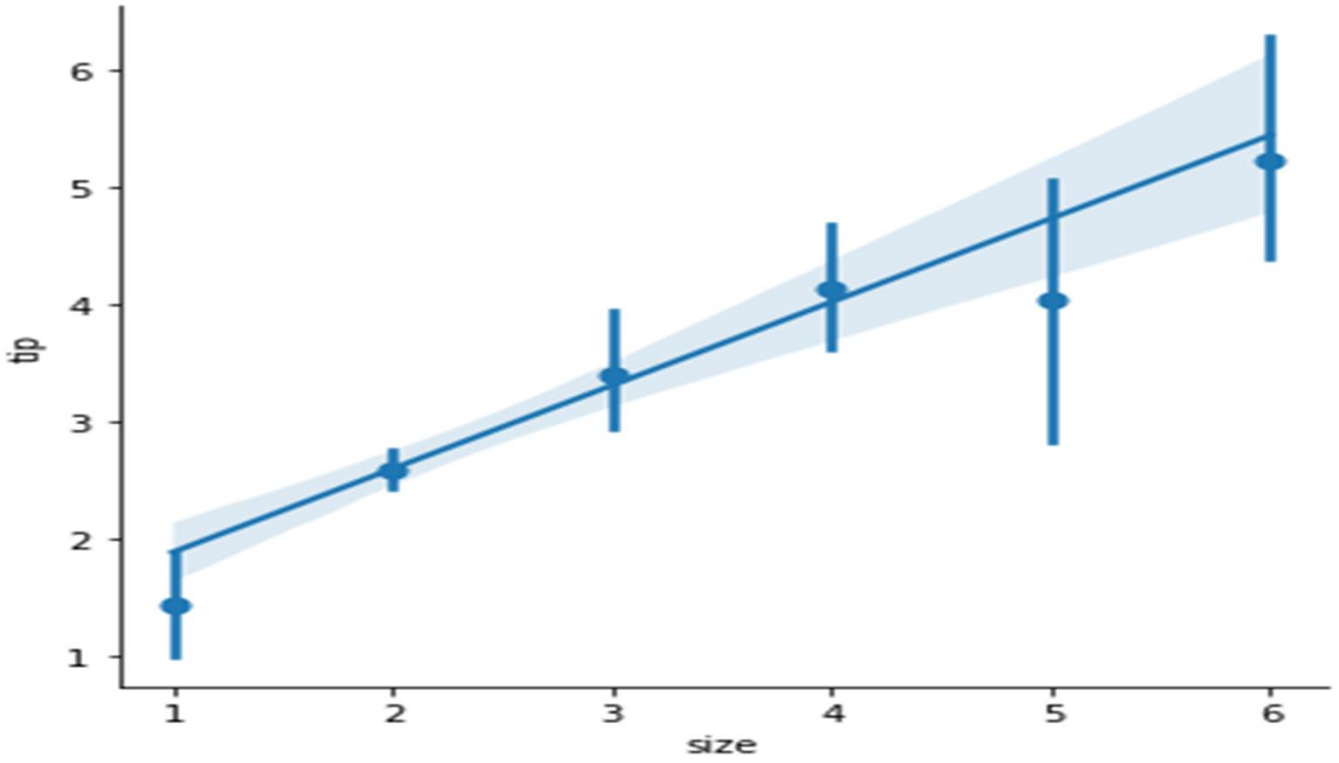


Figure 3.4: Linear regerssion plot with central tendency

3.2 Fitting different kinds of models

The simple linear regression model used above is very simple to fit, however, it is not appropriate for some kinds of datasets. The **Anscombe's quarter** dataset shows a few examples where simple linear regression provides an identical estimate of a relationship where simple visual inspection clearly shows differences. For example, in the first case, the linear regression **is a good model.**

```
anscombe = pd.read_csv("c:/data/anscombes.csv")

sns.lmplot(x="x", y="y", data=anscombe.query("dataset == 'I'"),ci=None,
scatter_kws={"s": 80});
```

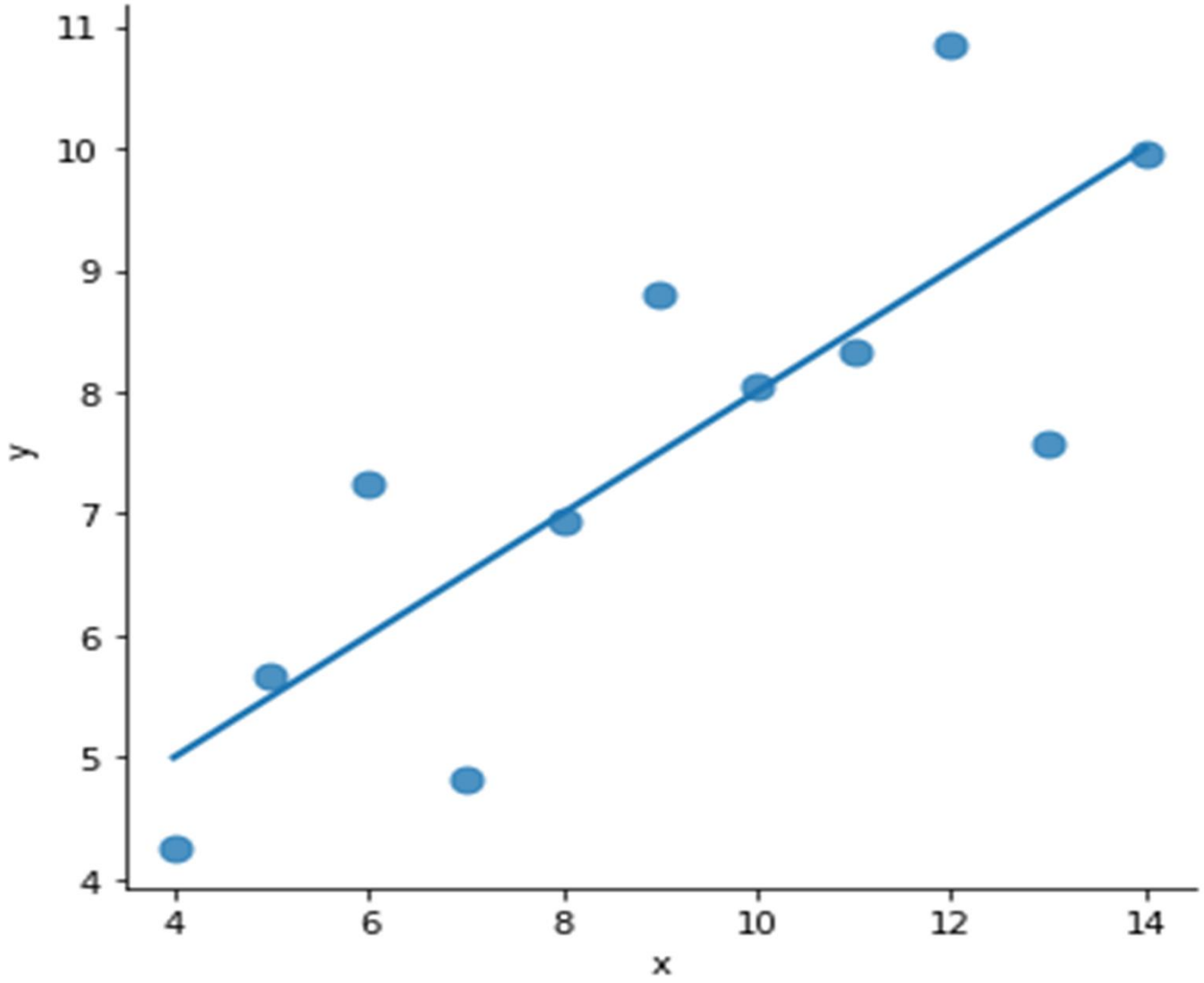


Figure 3.5: Linear regerssion plot with fisrt dataset Anscombe

✓ The linear relationship **in the second dataset** is the same, but the plot clearly shows that this **is not a good model.**

```
sns.lmplot(x="x", y="y", data=anscombe.query("dataset == 'II'"),
           ci=None, scatter_kws={"s": 80});
```

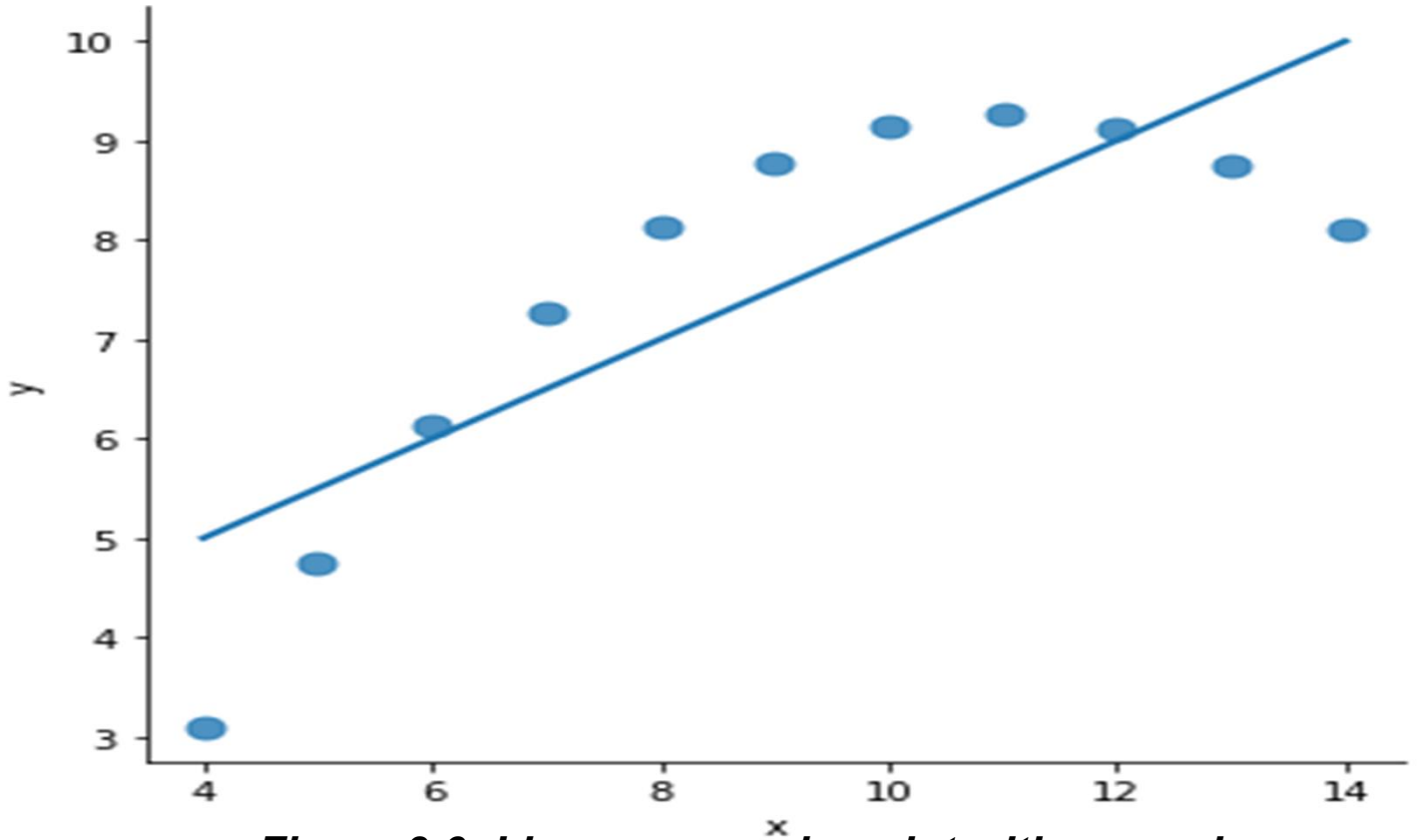


Figure 3.6: Linear regerssion plot with second dataset of Anscombe

✓ In the presence of these kind of higher-order relationships, **lmplot()** and **regplot()** can fit **a polynomial regression** model to explore simple kinds of nonlinear trends in the dataset.

```
sns.lmplot(x="x", y="y", data=anscombe.query("dataset =='II'"),order=2,
ci=None, scatter_kws={"s": 80});
```

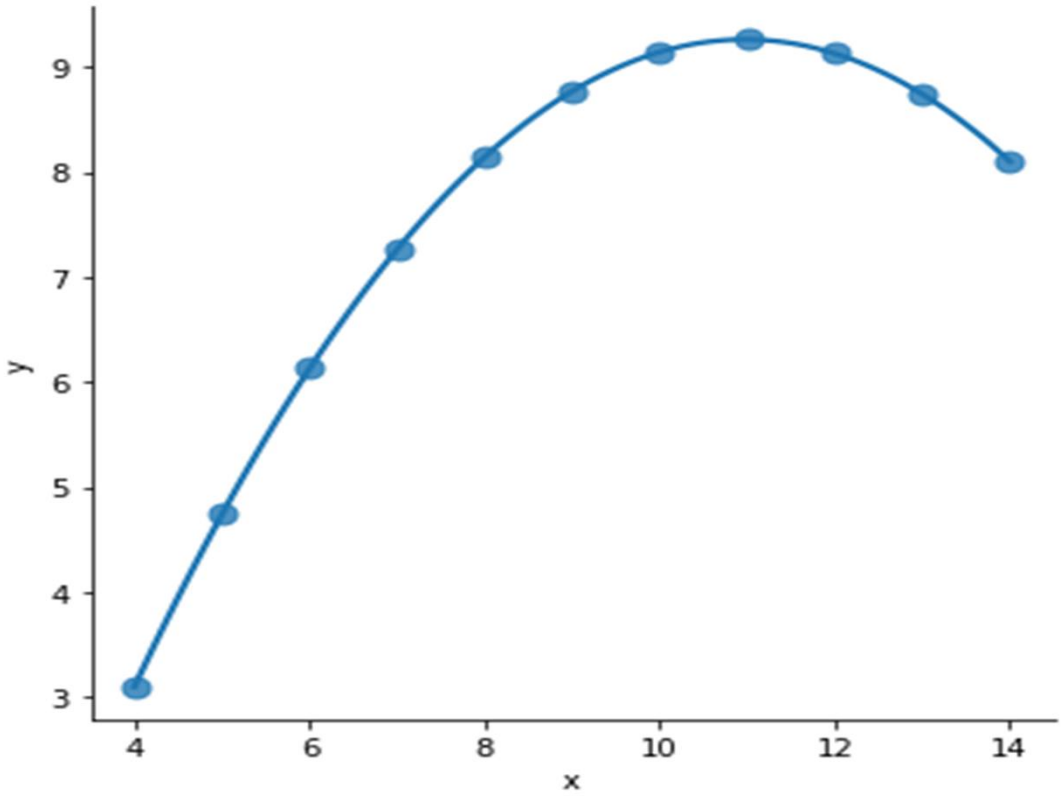


Figure 3.7: polynomial regerssion plot with second dataset of Anscombe

✓ A different problem is posed **by "outlier"** observations that deviate for some reason other than the main relationship under study.

sns.lmplot(x="x", y="y", data=anscombe.query("dataset == 'III'"),ci=None, scatter_kws={"s": 80});

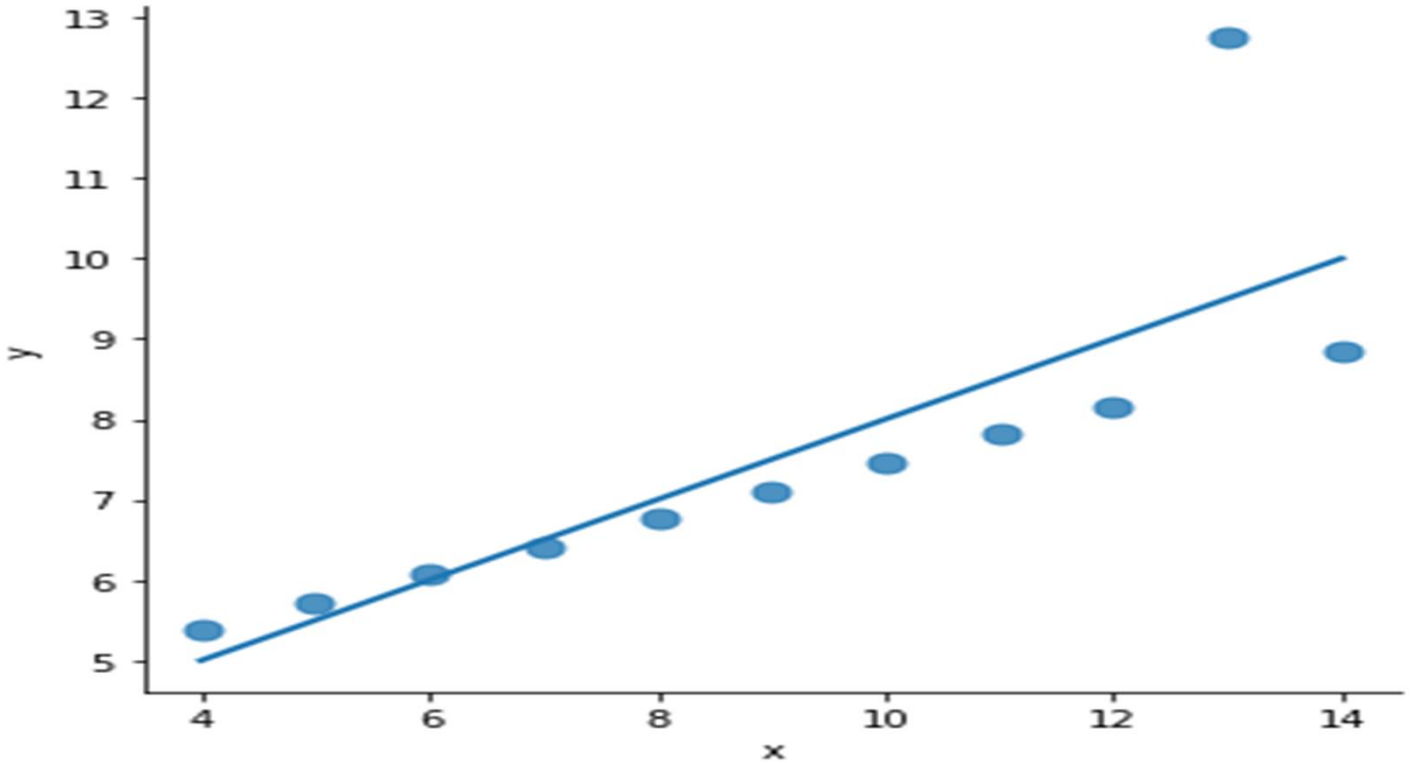


Figure 3.8: Linear regerssion plot with third dataset of Anscombe

✓ In the presence of outliers, it can be useful to fit **a robust regression**, which uses a different loss function to downweight relatively large residuals.

sns.lmplot(x="x", y="y", data=anscombe.query("dataset == 'III'"),robust=True, ci=None, scatter_kws={"s": 80});

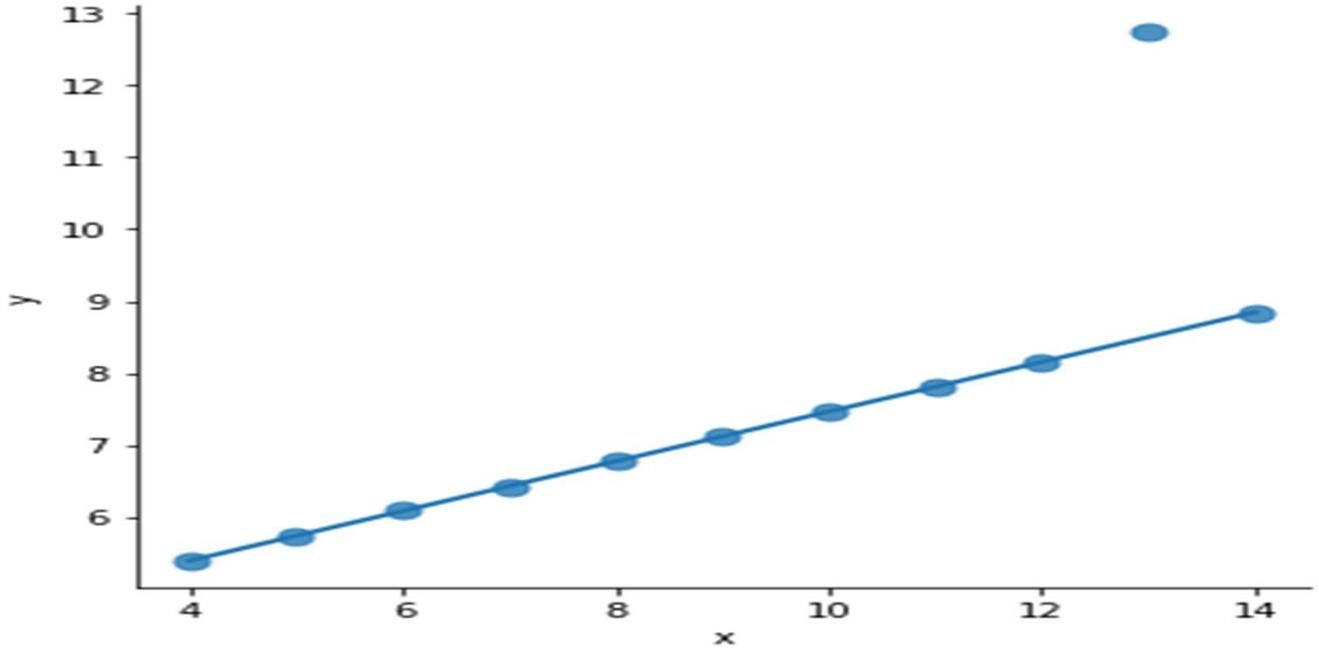


Figure 3.9: robust regerssion plot with third dataset of Anscombe

✓ When the variable is binary, simple linear regression also "works" but **provides implausible predictions:**

```
df["big_tip"] = (df.tip / df.total_bill) > .15
sns.lmplot(x="total_bill",y="big_tip", data=df,  y_jitter=.03);
```

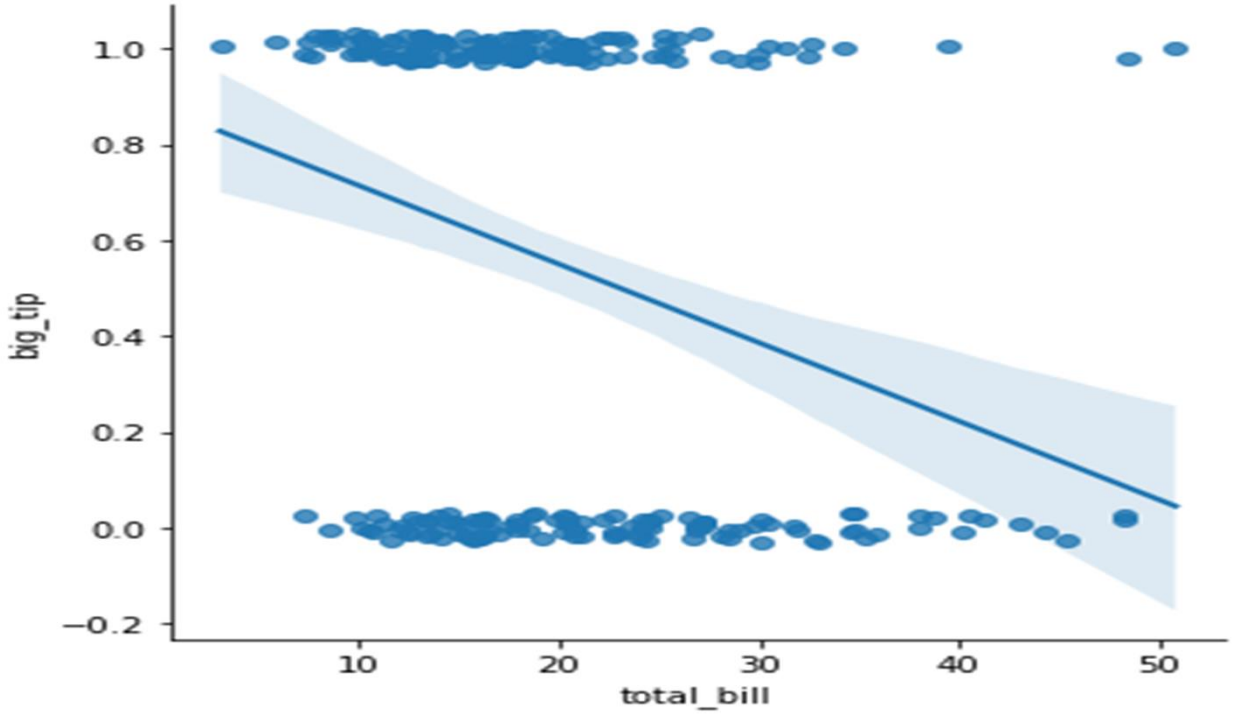


Figure 3.10: Linear regerssion plot with binary variable

✓ The solution in this case is to fit **a logistic regression**, such that the regression line shows the estimated probability of for a given value of :y = 1x

sns.lmplot(x="total_bill", y="big_tip", data=df, logistic=True, y_jitter=.03);

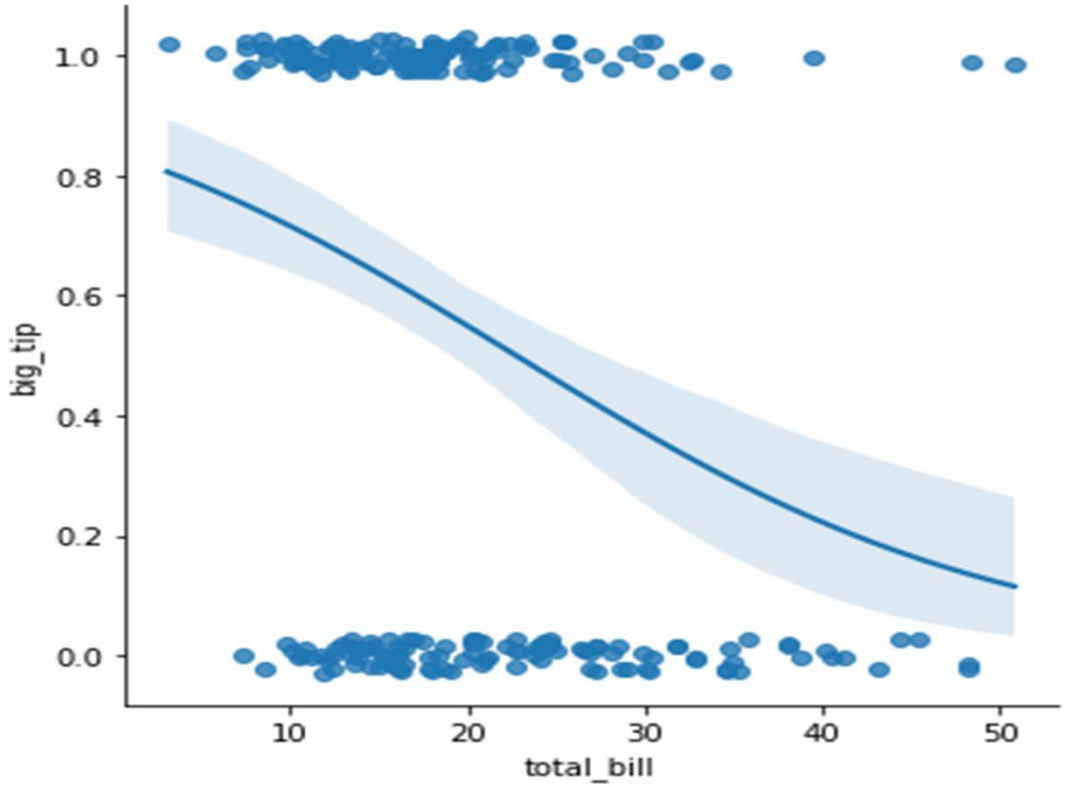


Figure 3.11: logistic regerssion plot

✓ The residplot() function can be a useful tool for checking whether the simple regression model **is appropriate** for a dataset. It fits and removes a simple linear regression and then plots the residual values for each observation. Ideally, these values should be randomly scattered around :y = 0

```
sns.residplot(x="x", y="y",
data=anscombe.query("dataset=='I'"),scatter_kws={"s": 80});
```

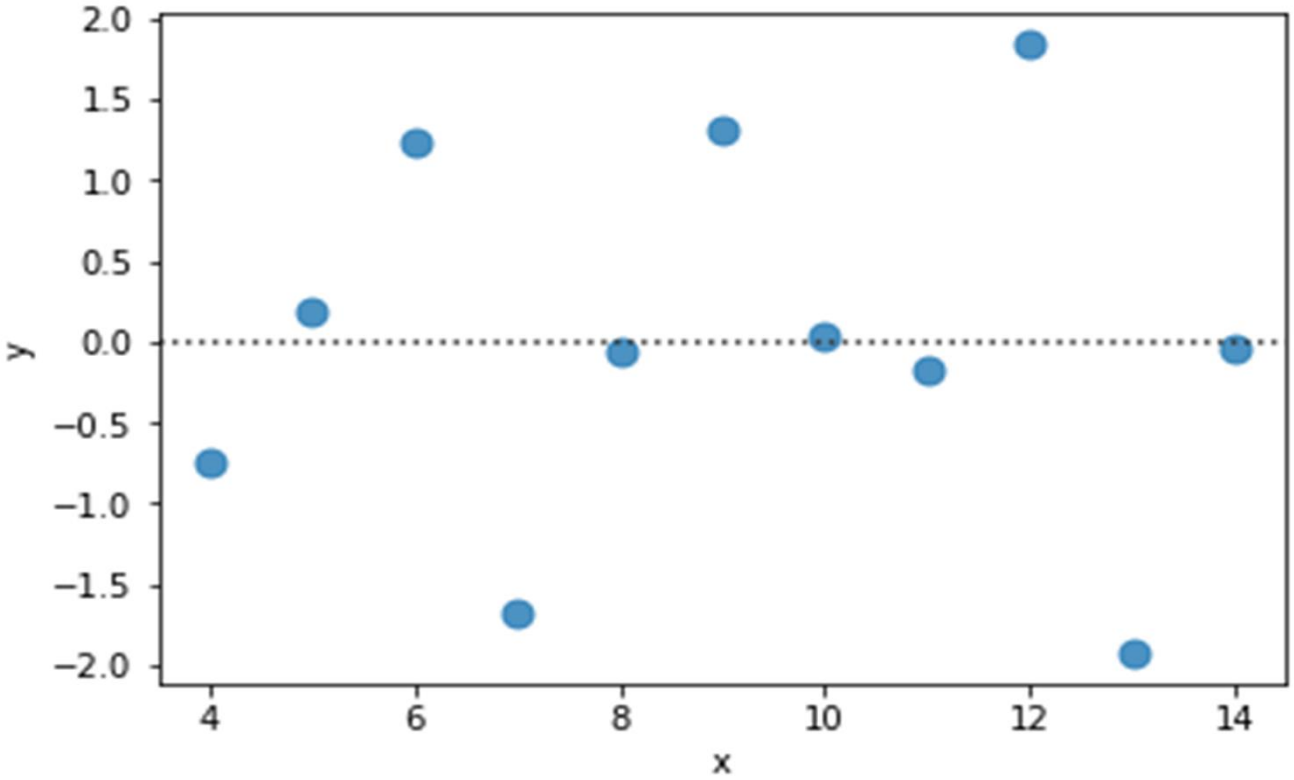


Figure 3.12: the randomly residual values plot

✓ If there is structure in the residuals, it suggests that simple linear regression **is not appropriate.**

```
sns.residplot(x="x", y="y", data=anscombe.query("dataset == 'II'"),
              scatter_kws={"s": 80});
```

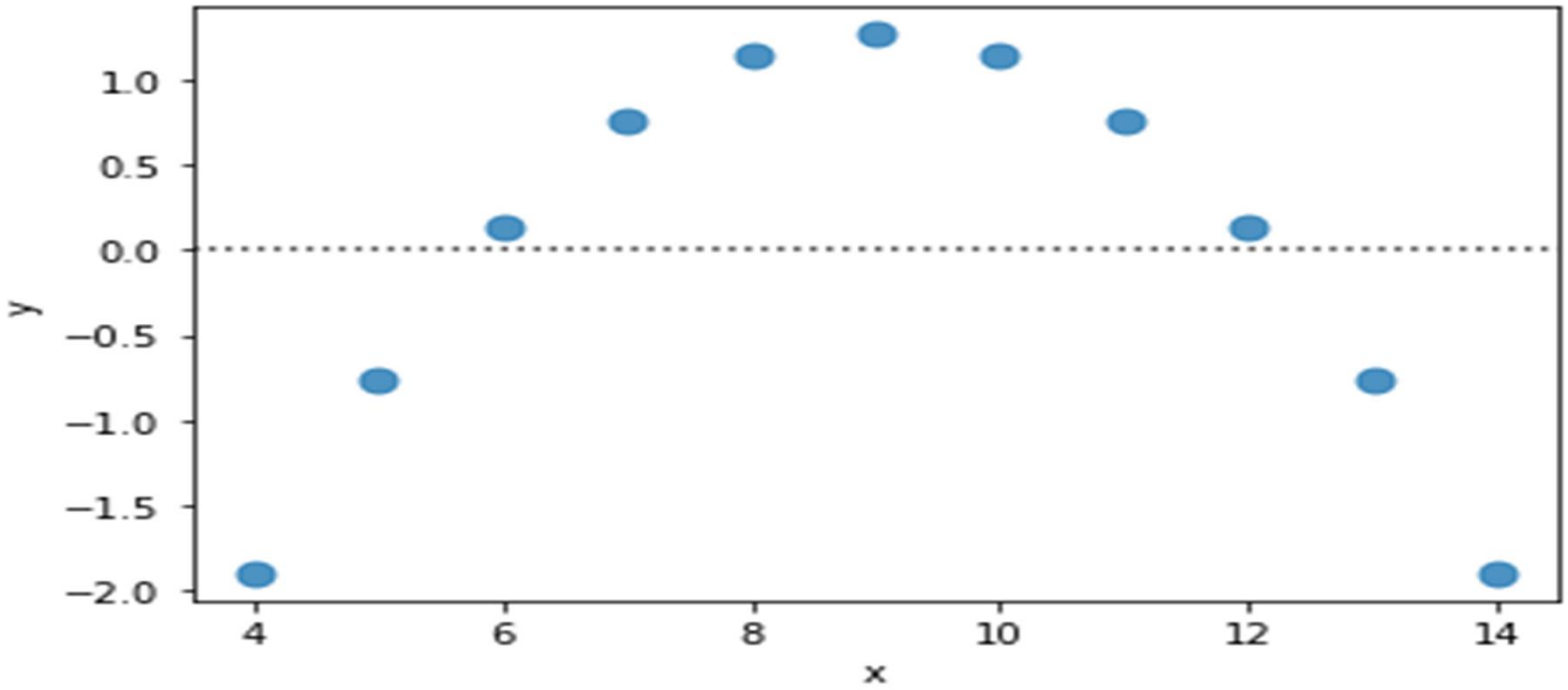
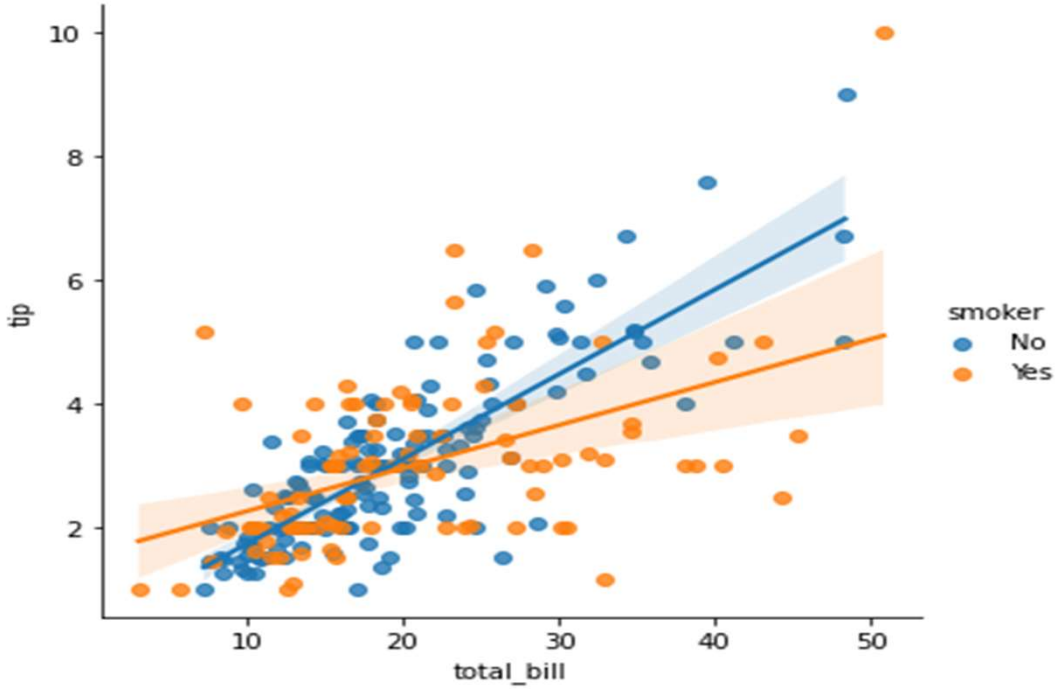


Figure 3.13: the structure residual values plot

3.3 Conditioning on other variables

The plots above show many ways to explore the relationship between a pair of variables. Often, however, a more interesting question is "how does the relationship between these two variables change as a function of a third variable?"

The best way to separate out a relationship is **to plot both levels on the same axes** and to **use color to distinguish them:**




```
sns.lmplot(x="total_bill",
y="tip",          hue="smoker",
data=df);
```

Figure 3.14: relationship between a pair variables a function the third variable

✓ In addition to color, it's possible to use different scatterplot markers to make plots the reproduce to black and white better. You also have full control over the colors used.

sns.lmplot(x="total_bill", y="tip", hue="smoker", data=df,markers=["o", "x"], palette="Set1");

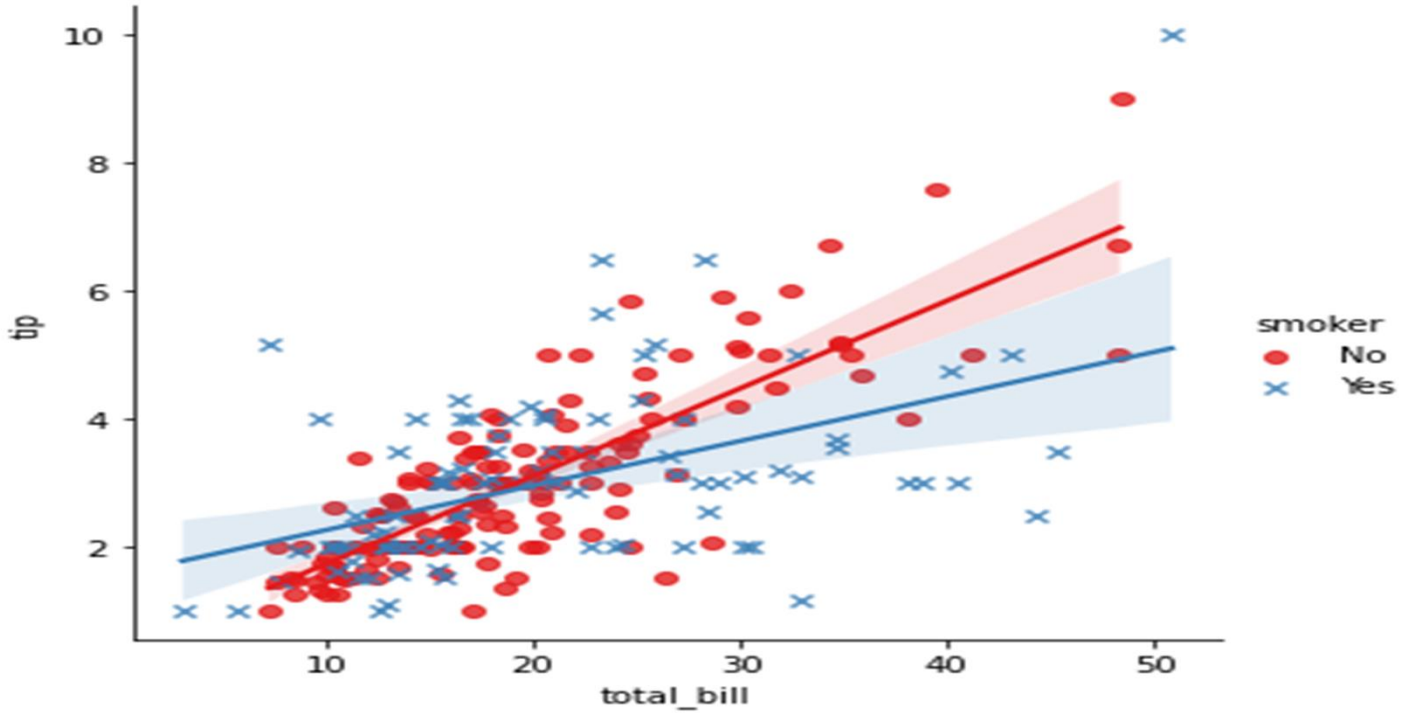


Figure 3.15: relationship between a pair variables a function the third variable to use diffrerent scatterplot markers

✓ To add another variable, you can draw multiple “facets” which each level of the variable appearing in the rows or columns of the grid.

sns.lmplot(x="total_bill", y="tip", hue="smoker", col="time", data=df);

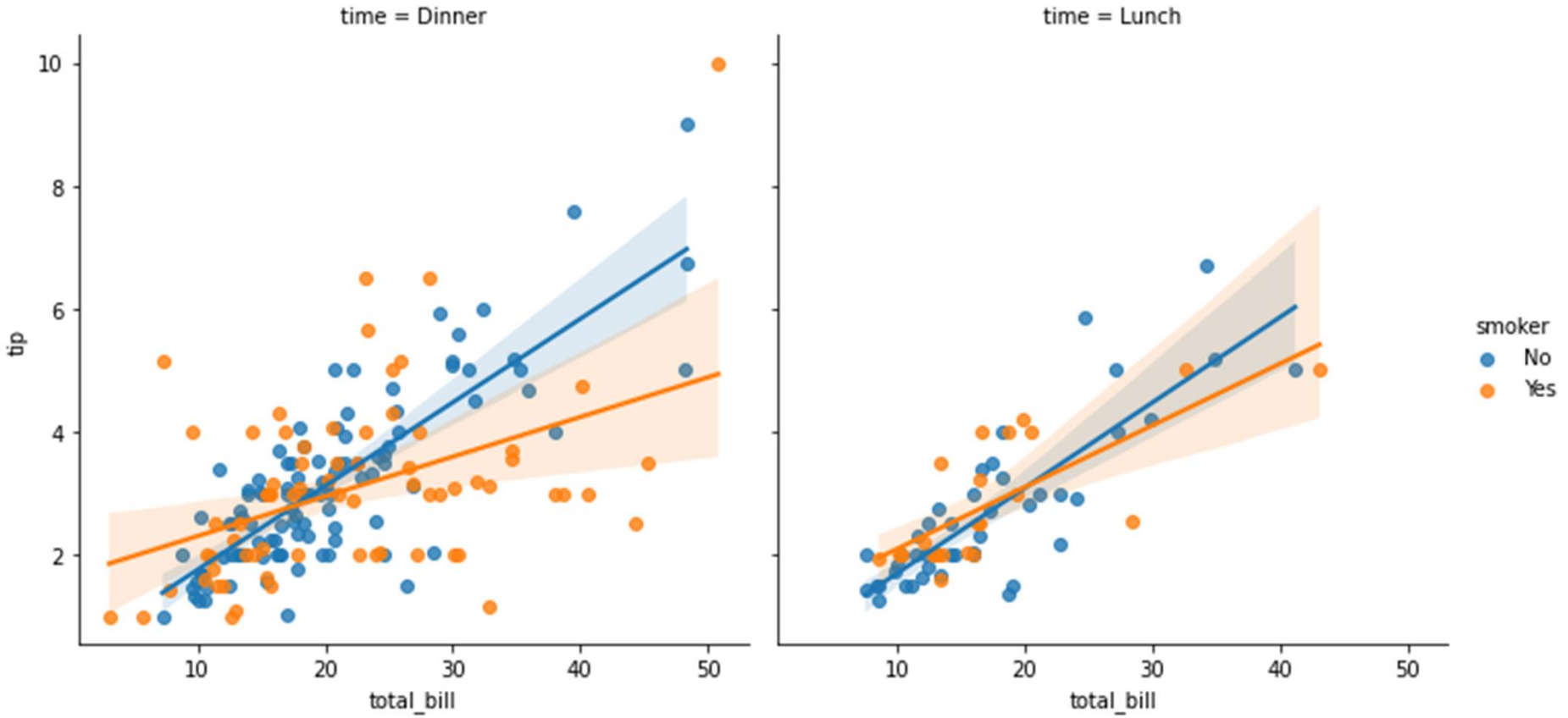


Figure 3.16: relationship between a pair variables a function the third and Forth variable

sns.lmplot(x="total_bill", y="tip", hue="smoker", col="time", row="sex", data=df);

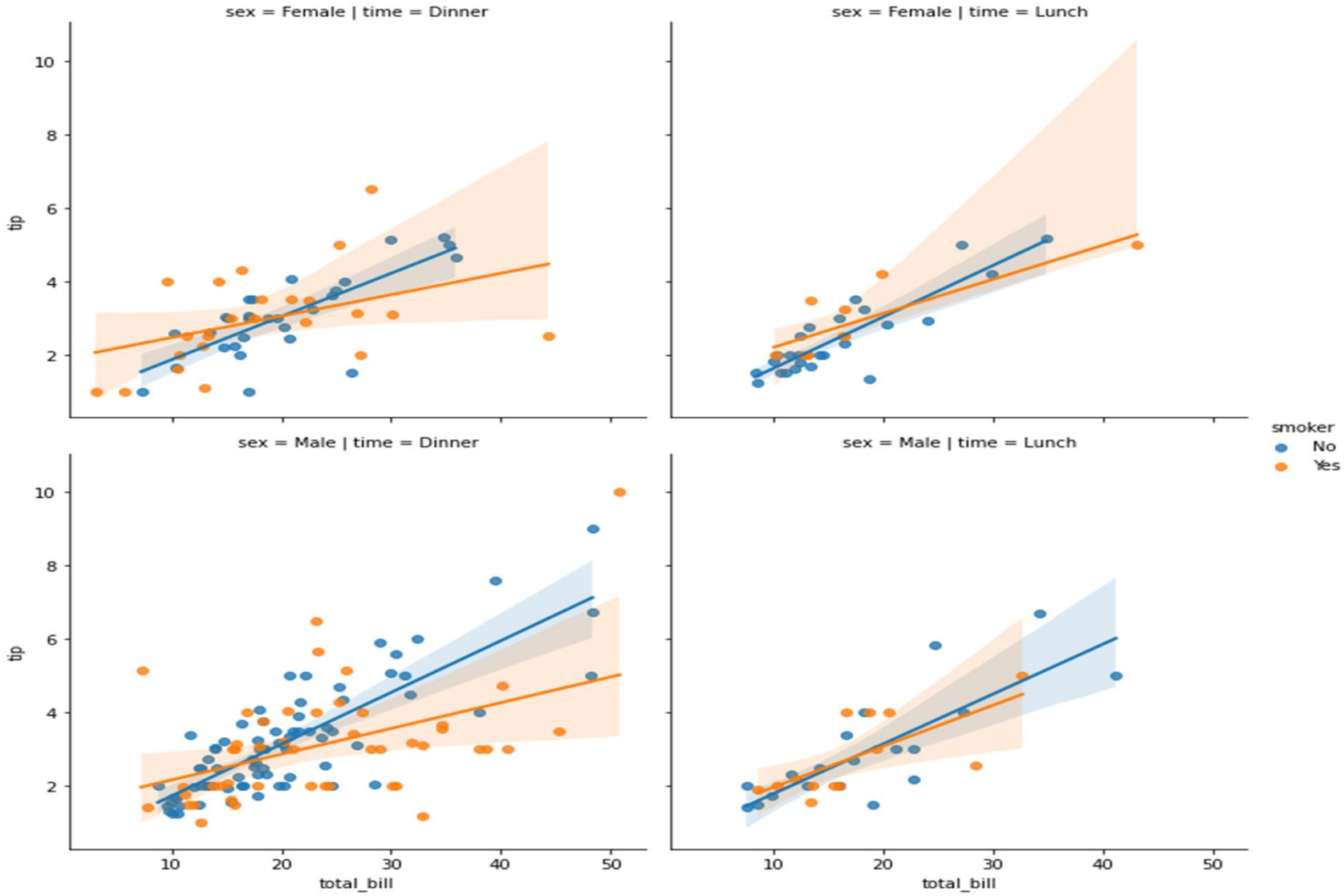


Figure 3.17: relationship between a pair variables a function the third and Forth and fifth variable

3.4 Plotting a regression in other contexts

✓ The first is the jointplot( )function that can use regplot() to show the linear regression fit on the joint axes by passing :kind="reg"

```
sns.jointplot(x="total_bill", y="tip", data=df, kind="reg");
```

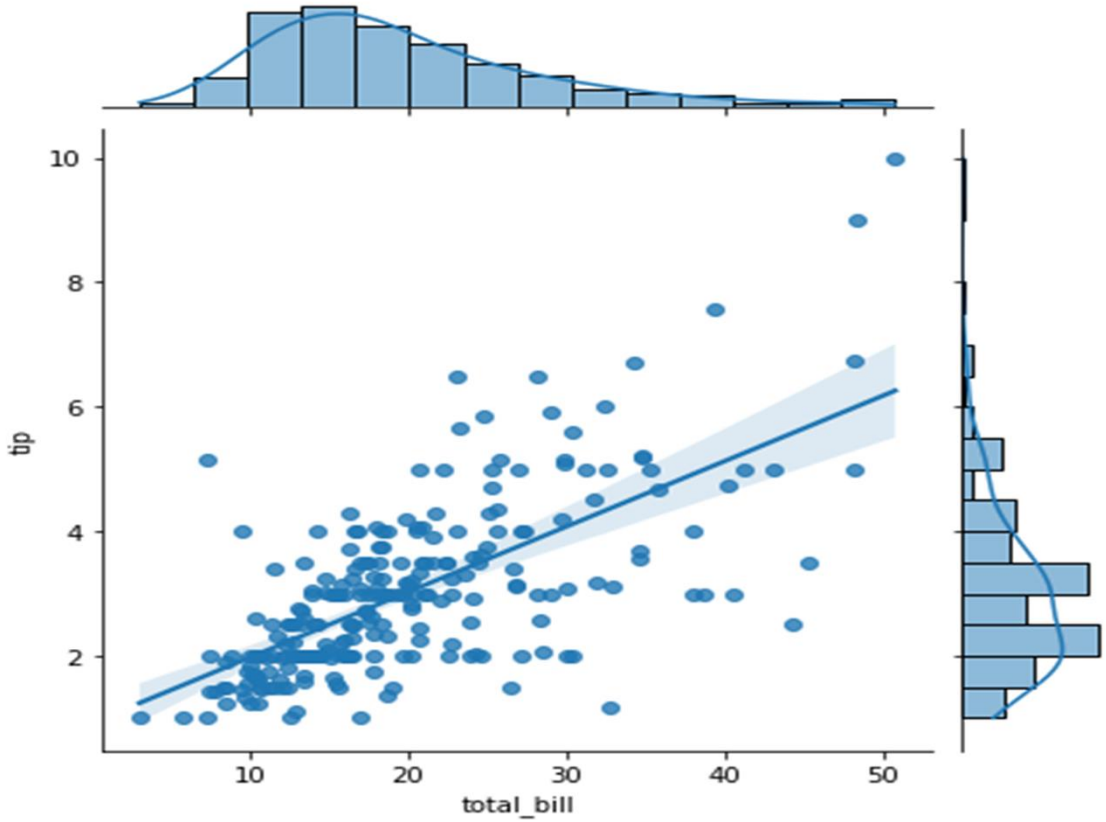


Figure 3.18: regression linear with jointplot()

✓ Using the pairplot() function to show the linear relationship between variables in a dataset.

```
sns.pairplot(df, x_vars=["total_bill", "size"], y_vars=["tip"],height=5, aspect=.8,
kind="reg");
```

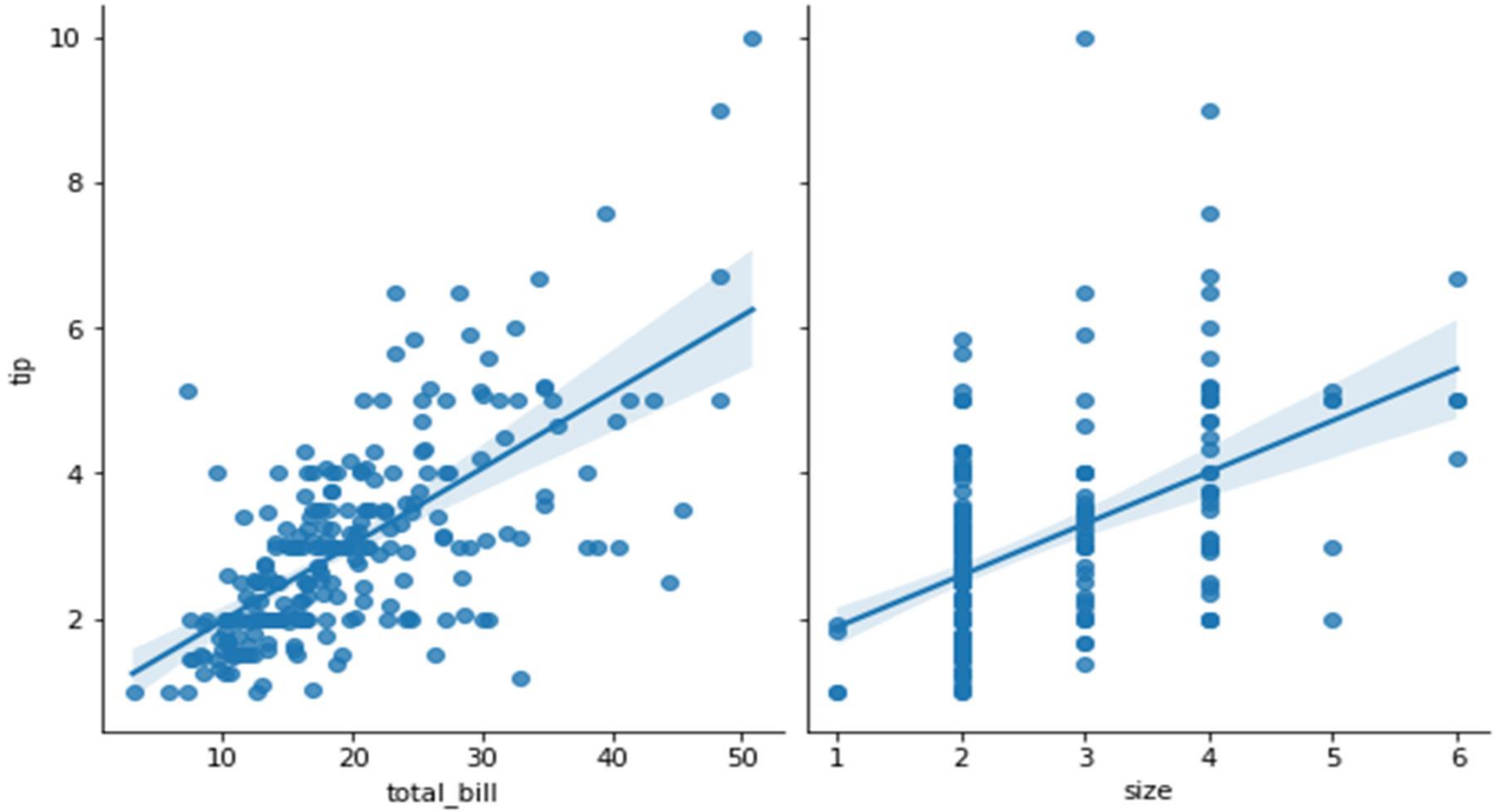


Figure 3.19: regression linear with pairplot()

✓ Like lmplot() but unlike jointplot(), conditioning on an additional categorical variable is built into pairplot() using the parameter:hue

sns.pairplot (df, x_vars=["total_bill", "size"], y_vars=["tip"],hue="smoker", height=5, aspect=.8, kind="reg");

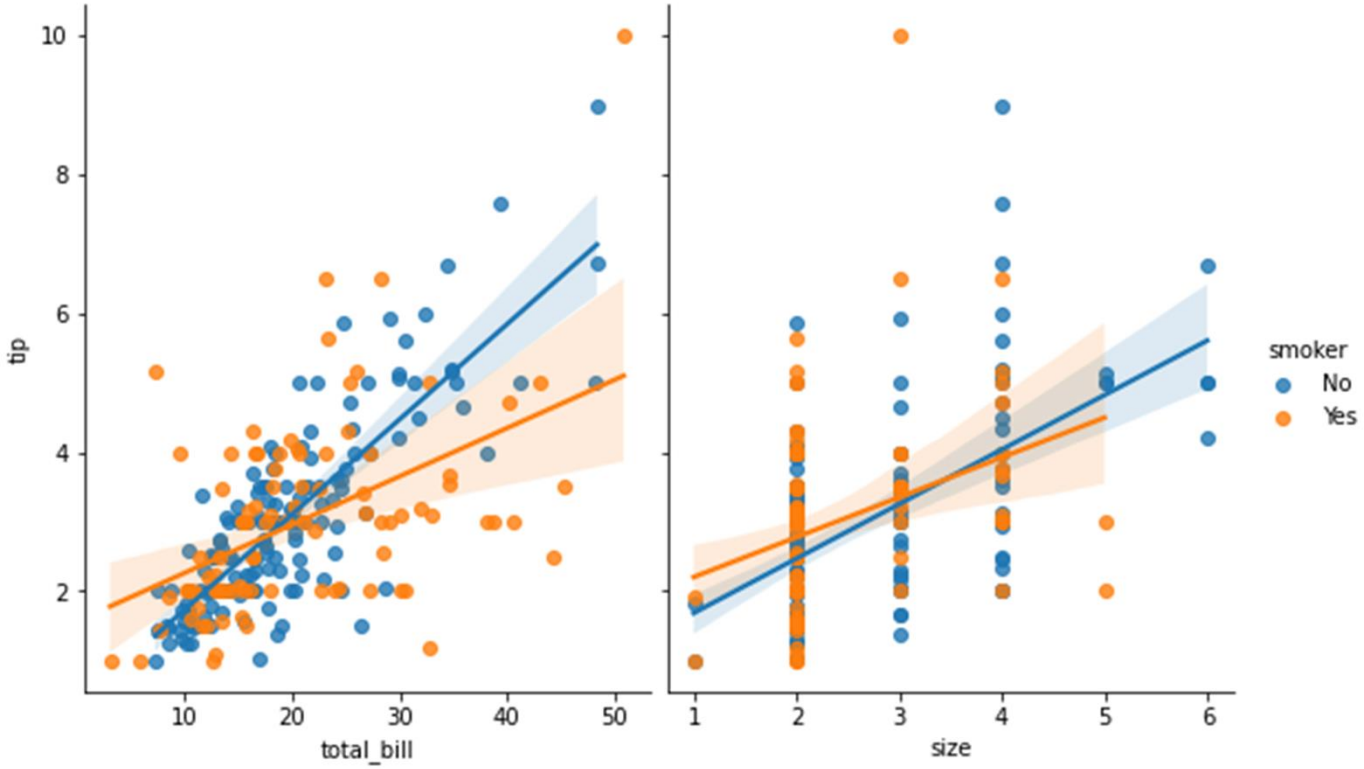
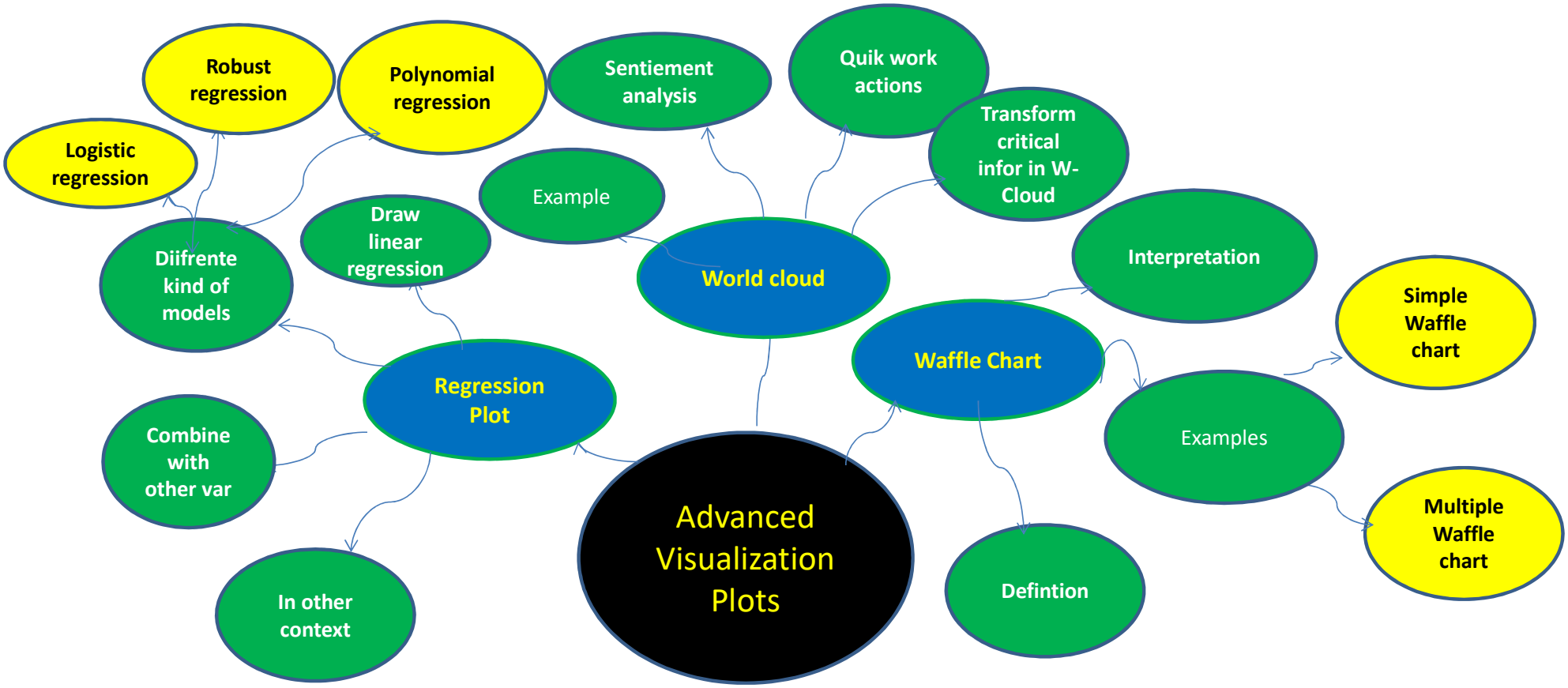


Figure 3.20: regression linear with pairplot() and parameter hue

6. Conclusion



References

https://fhsuguides.fhsu.edu/dataviz/partsof

https://python-charts.com/part-whole/waffle-chart-matplotlib/

https://www.pluralsight.com/guides/tableau-playbook-waffle-chart

https://towardsdatascience.com/text-analytics-101-word-cloud-and-sentiment-analysis-2c3ade81c7e8

https://python-charts.com/ranking/wordcloud-matplotlib/